

Searching Binary Embeddings

Three methods, one dataset

12 September 2026

Abstract

Exa stores document embeddings as short binary codes, groups them into clusters, and scores the documents of a few clusters with a lookup table. This report asks whether two extra index structures can reduce that scoring work: *Bitplanes*, which split a cluster by one sign bit at a time, and *Backward Walk*, which looks up documents by a short key. We derive the time, memory and recall of all three methods, use the formulas to choose each method’s best settings, and then measure every setting on one fixed dataset: one million MS MARCO passages and 200 queries. Only the index parameters change between runs. The formulas predict the recorded operation counts exactly and the measured search times within a few percent. On this dataset the original method is fastest at every recall target, and the calculations show why: on real embeddings, one sign bit removes only about 15% of the documents, while a cluster boundary removes about 98%. The same formulas also state the conditions under which Bitplanes or Backward Walk would win.

1 Introduction

1.1 Exa’s method, end to end

We build on the search pipeline that Exa describes in its engineering blog [1]. That post explains each technique in detail, so we only name the steps here and refer the reader to it. A document is embedded, the embedding is shortened by keeping its first values, which Matryoshka training makes possible [2], and each of the remaining d values is replaced by one bit: 1 if the value is positive, 0 if it is negative. This is Exa’s *binary quantization*. The binary codes are grouped into C clusters. When a query arrives, the system keeps the query in floating point, selects the P clusters whose centres are closest, and scores every document in those clusters with a lookup table that handles four bits at a time. The best documents can then be reranked with full vectors. Figure 1 shows the sequence; the blue box is the step that this report studies.

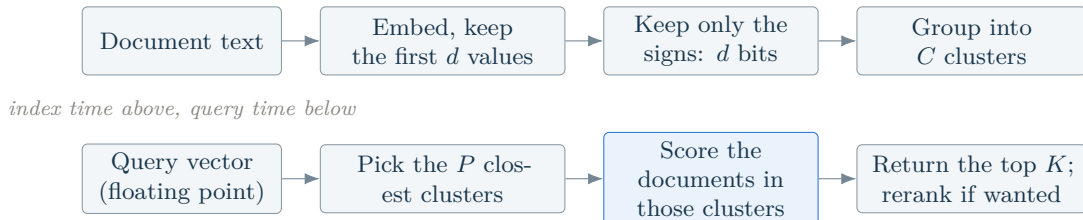


Figure 1: Exa’s pipeline as described in [1]. The top row happens once, when documents are indexed. The bottom row happens for every query. The blue box, scoring the documents of the opened clusters, is the step that the two new methods try to shorten. We call the unchanged pipeline **method A**, the original method.

Two numbers describe the scoring step. With N documents in C clusters of roughly equal size, opening P clusters means scoring about NP/C documents. For one million documents, 4096 clusters and one opened cluster, that is about 244 documents. Opening more clusters finds more of the true answers but scores more documents. The rest of this report is about that trade.

1.2 Idea background

The query tells us exactly which document code would score best. Take a small query with five values,

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

A document bit of 1 stands for +1 and a bit of 0 stands for −1; the score is the sum of q_i times the document’s ± 1 at each position. At the first position the query value is 0.7, so a document bit of 1 contributes +0.7 and a bit of 0 contributes −0.7. At the third position the query value is −0.4, so now a bit of 0 is better. Following this rule at every position gives the pattern 11001, and its score is $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$. No document can score higher.

Every other code loses some of that score. A document whose bit disagrees with the query sign at position i loses $2|q_i|$: instead of gaining $|q_i|$ it pays $|q_i|$. If we call the sum of the query weights at the disagreeing

positions the *penalty* p , then

$$S(q, b) = Q - 2p, \quad Q = \sum_i |q_i|, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

The code 11000 disagrees only at the last position, so its penalty is 0.3 and its score is $2.5 - 0.6 = 1.9$. The code 01001 disagrees only at the first position, which carries the largest weight; its score is $2.5 - 1.4 = 1.1$. Position matters: a mismatch on a large query value costs more than a mismatch on a small one.

This suggests a question. Method A scores every document in the opened clusters, even the ones that disagree with the query on its largest weights. *What if the index let us go straight to the documents whose codes are close to the preferred pattern, and skip the rest?* Two structures do this in different ways. Bitplanes split the documents by one sign bit at a time, largest query weight first. Backward Walk stores documents under a short key and visits the keys in order of increasing penalty. Both keep Exa’s scoring rule and its clusters; they change only the blue box of Figure 1.

We use the six documents in Table 1 as a running example, together with the query above and $K = 2$ results wanted. Scoring all six documents is what method A does. The best two are documents 0 and 1, with scores 2.5 and 1.9.

Table 1: Six example documents with five sign bits each, scored against $q = [0.7, 0.5, -0.4, -0.6, 0.3]$. The penalty adds the query weights at the positions where the document disagrees with the query sign.

ID	Code	Disagreeing positions (weights)	Score = $2.5 - 2p$
0	11001	none	2.5
1	11000	5 (0.3)	1.9
2	10100	2, 3, 5 (0.5, 0.4, 0.3)	0.1
3	01001	1 (0.7)	1.1
4	11101	3 (0.4)	1.7
5	00110	all five (2.5)	-2.5

1.3 Two new methods: Bitplanes and Backward Walk

1.3.1 Bitplanes

Method A stores each document as one row of bits. Bitplanes, which we call **method B**, additionally store the same bits sideways: one row per bit position, one column per document, inside each cluster. Figure 2 shows both layouts for the example. The sideways row for position 1 is 111010; it says that documents 0, 1, 2 and 4 have a 1 there. Such a row is called a *bitplane*. With 64-bit words, one bitplane for a cluster of n documents occupies $\lceil n/64 \rceil$ words, and the cluster needs d of them.

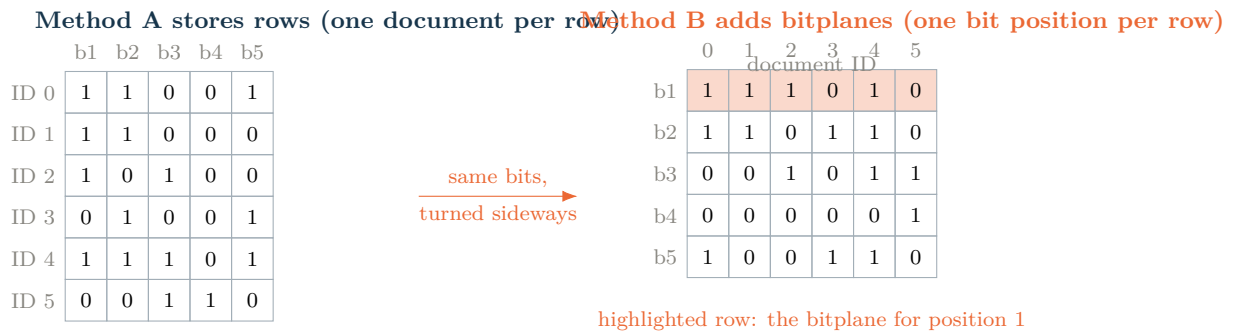


Figure 2: The two layouts of the same six codes. Method A keeps only the rows on the left. Method B keeps the rows for scoring and adds the bitplanes on the right for splitting. The highlighted bitplane belongs to position 1, which has the largest query weight.

Searching uses the bitplanes to narrow down the candidates before scoring (Figure 3). We start with a *mask* that marks every document in the cluster as a candidate: 111111. We take the position with the largest query weight, position 1 with 0.7, where the query prefers a 1. One AND between the mask and the bitplane for position 1 gives the documents that agree, 111010, which are documents 0, 1, 2 and 4. The remaining documents, 3 and 5, disagree at a position of weight 0.7, so every document in that group carries a penalty of

at least 0.7. We keep that group aside in a queue, labelled with its penalty, and continue with the agreeing group.

The next largest weight is position 4 (0.6, prefers 0). All four remaining documents have a 0 there, so nothing is removed. Position 2 (0.5, prefers 1) separates document 2 from documents 0, 1 and 4. When a group is small enough, which we call the *leaf size*, we stop splitting and score its documents with the same lookup table as method A. Scoring documents 0, 1 and 4 gives 2.5, 1.9 and 1.7; the best two are 2.5 and 1.9.

Now the queue decides whether any stored group needs attention. Equation 1 gives each group a ceiling: a group with penalty p cannot contain a score above $Q - 2p$. The group $\{2\}$ has penalty 0.5 and ceiling 1.5, and the group $\{3, 5\}$ has penalty 0.7 and ceiling 1.1. Both ceilings are below 1.9, the worse of the two scores we already hold, so neither group can change the answer. The search ends having scored three documents instead of six, with exactly the answer that method A returns.

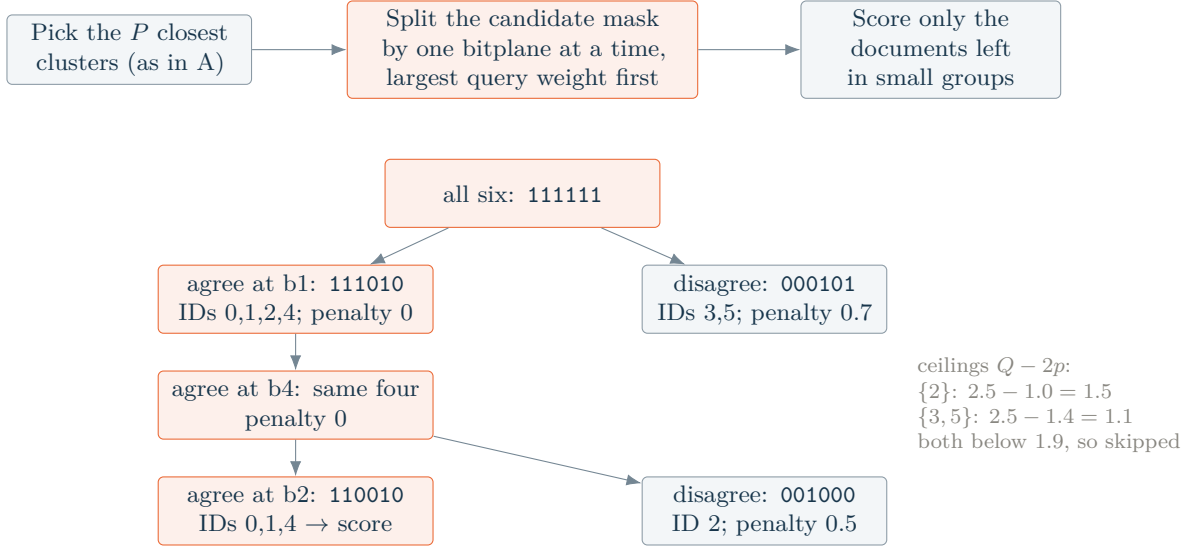


Figure 3: Method B on the example. Orange marks the changed step and the groups that the search follows. Each split keeps the disagreeing group in a queue with its penalty. After scoring the small group, the queued groups are dropped because their ceilings are below the second-best score already found.

```

bitplanes(query, opened_clusters, K, leaf_size, node_budget):
    table = score_table(query); best = top_k(K)
    order = positions sorted by |query value|, largest first
    queue = one full mask per opened cluster, penalty 0
    while queue is not empty:
        group = pop the group with the smallest penalty
        if Q - 2*group.penalty < best.worst_score(): break # no group can win
        if visited groups == node_budget: break # optional limit
        if group.count <= leaf_size or no positions remain:
            score every document in group.mask with table; offer to best
        else:
            agree, disagree = split group.mask by the bitplane of the next position
            push agree (same penalty); push disagree (penalty + |query value|)
    return best
  
```

Two settings control method B. The *leaf size* decides when a group is small enough to score. The *node budget* caps how many groups the search may visit; with no budget, the ceiling rule alone stops the search, and the answer is then the same as method A’s within the opened clusters. Both children of a split are as wide as the whole cluster: a mask of $\lceil n/64 \rceil$ words stays that wide even when only a few bits remain set. This is the main cost of method B, and Section 2 counts it.

1.3.2 Backward Walk

The second structure, which we call **method C**, makes the “go straight to the preferred pattern” idea literal. Choose h fixed bit positions and read a document’s bits at those positions as a short *key*. Inside each cluster, sort the documents by key so that documents with the same key sit next to each other, and keep a small *directory* that maps each key to the start and length of its row range. Figure 4 shows this for the example

with positions 2 and 3 as the key.

Method C sorts the rows by key and adds a directory

	b1	b2	b3	b4	b5		
ID 2	1	0	1	0	0	key 01	directory (key 00 is absent: no document has it)
ID 5	0	0	1	1	0		
ID 0	1	1	0	0	1	key 10	
ID 1	1	1	0	0	0		
ID 3	0	1	0	0	1	key 11	
ID 4	1	1	1	0	1		

key	start	count
01	0	2
10	2	3
11	5	1

Figure 4: The index of method C for the example, with bit positions 2 and 3 as the key (highlighted). Rows are stored in key order, so the documents of one key form one contiguous range. The directory holds one entry per key that occurs. The row IDs double as the list of documents, so no separate posting list is needed.

At query time we form the query’s preferred key from the same positions. The query prefers a 1 at position 2 and a 0 at position 3, so its key is 10. A directory lookup returns documents 0, 1 and 3, and we score them: 2.5, 1.9 and 1.1. If this were enough we would stop; but the best documents may have a different key, so the search must be able to move on. It does so by walking away from the preferred key one flipped position at a time, cheapest flip first, which is why we call it a backward walk (Figure 5). Flipping position 3 (weight 0.4) gives key 11 with penalty 0.4; flipping position 2 (weight 0.5) gives 00 with penalty 0.5; flipping both gives 01 with penalty 0.9. As in method B, a key with penalty p has ceiling $Q - 2p$. The next key, 11, has ceiling $2.5 - 0.8 = 1.7$, which is below the second-best score 1.9 that we already hold. The walk stops after scoring three documents, again with method A’s answer.

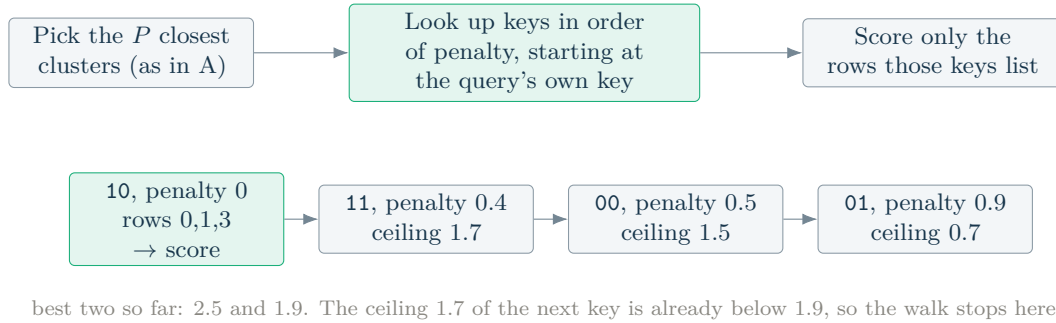


Figure 5: Method C on the example. Green marks the changed step. Keys are visited in increasing penalty; each visit is one directory lookup per opened cluster. The walk stops as soon as the next key’s ceiling cannot beat the K th best score.

```

backward_walk(query, opened_clusters, K, key_positions):
    table = score_table(query); best = top_k(K)
    keys = every key, ordered by penalty against the query's preferred key
    for key, penalty in keys:
        if Q - 2*penalty < best.worst_score(): break # no key can win
        for cluster in opened_clusters:
            rows = cluster.directory.lookup(key) # may be empty
            score each row with table; offer to best
    return best
  
```

Method C has one main setting, the key width h . There are 2^h possible keys. A wide key makes each key’s row range small but multiplies the number of keys the walk may have to visit; a narrow key does the opposite. The key positions are fixed when the index is built, so they cannot follow each query’s largest weights the way method B’s split order does. The positions the enumeration visits are ordered by the query weights at those h positions only.

Both new methods return exactly method A’s answer within the opened clusters when they run to completion, because the ceiling rule never drops a group or key that could still hold a top- K document. They can be faster only if the ceilings let them skip enough documents to pay for the extra reads. Section 2 counts that work,

Section 3 uses the counts to choose each method’s settings on the dataset, and Section 4 measures the result.

2 Preliminary calculations

This section counts what each method does for one query. The counts use the quantities in Table 2. Section 4 records the same counts from the running code, so every formula here is checked against the implementation.

Table 2: Quantities used in the calculations, with the values of the experiment where fixed.

Symbol	Meaning	In the experiment
N	documents in the collection	1,000,000
d	bits per document code	256
K	results returned per query	100
C, P	clusters in the index; clusters opened for a query	chosen per method
n_c	documents in cluster c	from the data
$W_c = \lceil n_c/64 \rceil$	64-bit words in one mask or bitplane of cluster c	from the data
L	leaf size of method B: a group this small is scored	chosen
h	key width of method C, in bits	chosen
$Q = \sum_i q_i $	total query weight; the best possible score	per query
g	gap: the penalty of the K th best document, $(Q - S_K)/2$	per query
$c.$	milliseconds per counted operation (measured in Section 4)	measured

2.1 Time complexity

Every method first selects P clusters, which costs T_{route} and is the same for all three at the same C and P . We therefore concentrate on what happens inside the opened clusters.

2.1.1 Original method

Method A scores every document in the opened clusters. Writing F_A for that number,

$$F_A = \sum_{c \text{ opened}} n_c \approx \frac{NP}{C}, \quad T_A = T_{\text{route}} + c_{\text{call}} + c_{\text{score}} F_A.$$

One score reads $d/4 = 64$ table entries and offers the result to a heap that keeps the best K . The constant c_{score} covers both. c_{call} is the fixed cost of one call: building the 64×16 lookup table and returning the sorted result. For one million documents in 4096 clusters, opening one cluster scores about 244 documents; opening 1330 clusters, which Section 3 shows is needed for 99% recall, scores about 322,000.

2.1.2 Bitplanes

Method B replaces some scoring by mask work. A split of a group in cluster c reads W_c words of one bitplane and writes two child masks of W_c words each. If the search performs J_c splits in cluster c and scores E_c groups there, it visits

$$V_s = \sum_c J_c W_c \text{ split words}, \quad V_l = \sum_c E_c W_c \text{ leaf words}.$$

The leaf words are read to find which bits are still set before scoring those documents. Both counts use the full width W_c : a group that has shrunk to 80 documents inside a cluster of 6,400 still has a mask of 100 words, and the next split reads all 100. With F_B documents scored and G groups taken from the queue,

$$T_B = T_{\text{route}} + c_{\text{call}} + c_{\text{score}} F_B + c_{\text{split}} V_s + c_{\text{leaf}} V_l + c_{\text{group}} G.$$

c_{group} prices the queue operations and the allocation of the two child masks. When $L \geq n_c$ for every opened cluster, no split happens: $V_s = 0$, $V_l = \sum_c W_c$, $G = P$, and $F_B = F_A$. Method B then does method A’s work plus one pass over the masks.

2.1.3 Backward Walk

Method C replaces scoring by directory lookups. Each key it visits costs one lookup in each opened cluster, and an absent key still costs the lookup. With E keys visited,

$$H = EP \text{ lookups}, \quad T_C = T_{\text{route}} + c_{\text{call}} + c_{\text{score}} F_C + c_{\text{lookup}} H + c_{\text{key}} E.$$

c_{key} prices generating the next key in penalty order. If the ceiling rule never stops the walk, it visits every key: $E = 2^h$ and $F_C = F_A$. With $h = 8$ and 1330 opened clusters that is $256 \times 1330 \approx 340,000$ lookups on top of method A's scoring.

2.2 Memory complexity

We separate the index, which is held permanently, from the working memory of one query.

2.2.1 Original method

Each code occupies $d/8 = 32$ bytes and each document ID 8 bytes:

$$M_A = N(32 + 8) \text{ bytes} = 40 \text{ MB at } N = 10^6,$$

plus the cluster centres, $4Cd$ bytes (4 MB for $C = 4096$). A query needs the lookup table (64×16 doubles, 8 KB) and the heap of K results.

2.2.2 Bitplanes

Method B keeps method A's rows and adds d bitplanes per cluster:

$$M_B = M_A + 8d \sum_{c=1}^C W_c \text{ bytes.}$$

For one cluster of one million documents, $\sum_c W_c = 15,625$ and the planes add $8 \times 256 \times 15,625 = 32$ MB, making 72 MB. With 4096 clusters of about 238 documents each mask is padded to whole words, so $\sum_c W_c$ rises to about 16,400 and the planes to about 33.6 MB. During a query, every group waiting in the queue holds a mask of W_c words; the peak is the largest number of masks alive at one time, times their widths, times 8 bytes.

2.2.3 Backward Walk

Method C reorders the rows within each cluster and keeps their IDs, so it stores the same 40 MB as method A. It adds a directory entry of 20 bytes (a 4-byte key, an 8-byte start and an 8-byte count) for each key that actually occurs:

$$M_C = M_A + 20U \text{ bytes,} \quad U \leq \min(N, C 2^h).$$

With $h = 8$ and 4096 clusters, U is at most about one million entries, or 20 MB; the hash table that holds them adds its own overhead. The query's working memory is the queue of keys still to visit. Figure 6 compares the three.

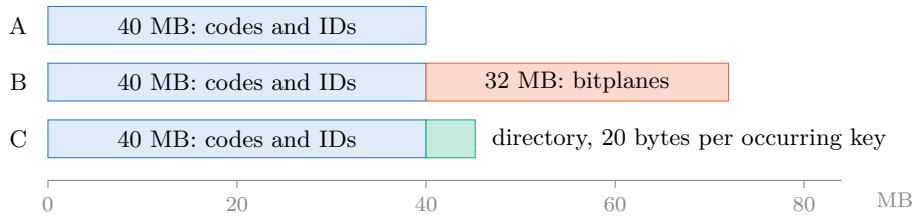


Figure 6: Index memory for one million 256-bit codes. Cluster centres, the hash table overhead of C, and the working memory of a query are additional. The bitplane bar is for one cluster; many small clusters add a little padding.

2.3 Recall estimation

The reference answer for a query is the top K when every document is scored. Recall is the share of those K IDs that a method returns:

$$R = \frac{\text{reference IDs returned}}{K}.$$

Returning 99 of the 100 reference documents is a recall of 99%. This measures agreement with the exact binary score, not whether a passage is a good answer to the question.

2.3.1 Original method

Method A scores every document in the opened clusters, so it can miss a reference document only if that document's cluster was not opened. Let $r_{\text{route}}(C, P)$ be the share of reference documents that lie in the P opened clusters, averaged over queries. Then

$$R_A = r_{\text{route}}(C, P).$$

This is a property of the clustering and the data, and Section 3 measures it directly.

2.3.2 Bitplanes

Method B can lose a document in a second way: by never scoring the group that holds it. Writing r_B for the share of reference documents inside the opened clusters that method B actually returns,

$$R_B = r_{\text{route}}(C, P) \times r_B.$$

Without a node budget, $r_B = 1$: the ceiling rule drops a group only when its ceiling $Q - 2p$ is below the K th best score already found, and such a group cannot contain a top- K document. In terms of the gap g of Table 2, a group is skipped exactly when its penalty exceeds g . In the running example the second-best score is 1.9, so $g = (2.5 - 1.9)/2 = 0.3$, and the group $\{2\}$ with penalty 0.5 is skipped. With a node budget, r_B falls below one, and by how much depends on where the reference documents sit in the split tree. That has no closed form; Section 4 measures it.

2.3.3 Backward Walk

The same reasoning gives

$$R_C = r_{\text{route}}(C, P) \times r_C.$$

$r_C = 1$ when the walk runs until the ceiling rule stops it, because a key is skipped only when its penalty exceeds g . A limit on lookups or on scored documents makes $r_C < 1$.

The gap g is the quantity that decides whether the new methods can skip anything. If no reachable group or key has a penalty above g , both methods score every document that method A scores and add their own work on top. Section 3 measures g on the dataset and compares it with the query weights that a split or a key flip can accumulate.

2.4 Summary

Tables 3, 4 and 5 collect the formulas. The routing cost T_{route} and the constants c . are measured in Section 4.

Table 3: Time for one query. F counts scored documents; V_s, V_l count 64-bit words read while splitting and while collecting a group's documents; G counts groups taken from the queue; H counts directory lookups and E the keys visited.

Method	Time formula
A: original	$T_A = T_{\text{route}} + c_{\text{call}} + c_{\text{score}}F_A$, with $F_A = \sum_{\text{opened}} n_c \approx NP/C$
B: bitplanes	$T_B = T_{\text{route}} + c_{\text{call}} + c_{\text{score}}F_B + c_{\text{split}}V_s + c_{\text{leaf}}V_l + c_{\text{group}}G$, with $V_s = \sum_c J_c W_c$, $V_l = \sum_c E_c W_c$
C: backward walk	$T_C = T_{\text{route}} + c_{\text{call}} + c_{\text{score}}F_C + c_{\text{lookup}}H + c_{\text{key}}E$, with $H = EP$ and $E \leq 2^h$

Table 4: Index memory, for $d = 256$. U is the number of keys that occur in the data.

Method	Index memory	Working memory per query
A: original	$M_A = 40N$ bytes, plus $4Cd$ for cluster centres	lookup table and heap of K
B: bitplanes	$M_A + 8d \sum_c W_c$	table, heap, and one $8W_c$ -byte mask per queued group
C: backward walk	$M_A + 20U$, plus hash-table overhead	table, heap, and the queue of pending keys

Table 5: Recall. r_{route} is the share of reference documents inside the opened clusters; r_B and r_C are the shares of those that the local search returns.

Method	Recall	When the local search loses nothing ($r = 1$)
A: original	$r_{\text{route}}(C, P)$	always: every opened document is scored
B: bitplanes	$r_{\text{route}}(C, P) r_B$	no node budget; groups are dropped only when penalty $> g$
C: backward walk	$r_{\text{route}}(C, P) r_C$	no lookup or candidate limit; keys are dropped only when penalty $> g$

3 Deriving optima

We now choose each method’s settings for the dataset of Section 4: one million MS MARCO passages embedded with Nomic v1.5, cut to 256 values and stored as 256 bits, with 200 queries and $K = 100$. The requirement is the same for all three methods: return at least a target share of the true top 100 (we use 80%, 90%, 95% and 99%), keep the index within 32 GB of RAM, and be as fast as possible. Memory turns out not to constrain anything here: the largest index in Section 2 is under 100 MB. The choice is therefore made on time, and each method may pick its own cluster count C , opened clusters P and local settings. We treat the methods one at a time and combine the results in Section 3.4.

3.1 Original method

Method A has two settings, C and P , and one formula, $T_A = T_{\text{route}}(C, P) + c_{\text{call}} + c_{\text{score}} F_A(C, P)$. The recall requirement removes one degree of freedom: for each C , P must be the smallest number of clusters that contains the target share of the reference documents. That number comes from the data. We rank every cluster by its distance to each query, note the rank of the cluster that holds each reference document, and read off how many clusters must be opened to reach each target. Figure 7 shows the result as recall against documents opened, and Table 6 lists the values at the four targets.

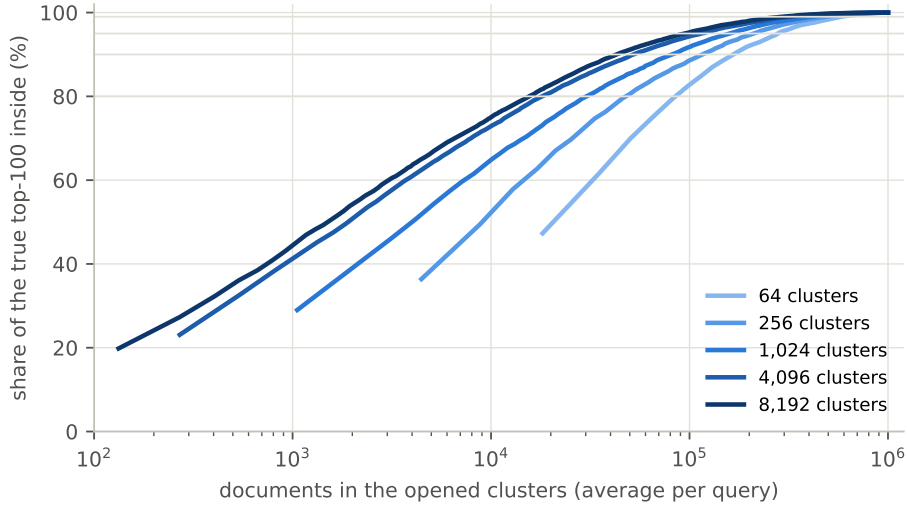


Figure 7: Share of the true top-100 that lies in the opened clusters, against the number of documents those clusters contain, for five cluster counts on the dataset. Horizontal lines mark the four recall targets. More clusters reach a target with fewer documents, because the opened region fits the query more tightly.

Two effects pull in opposite directions as C grows. The scoring term $c_{\text{score}} F_A$ falls, because Table 6 shows fewer documents for the same recall: at 99%, 574,000 documents with 64 clusters against 296,000 with 8,192. The routing term rises, because selecting clusters compares the query with all C centres, $256C$ multiplications, and then sorts out the best P . Writing $T_{\text{route}} \approx c_{\text{centre}} C + c_{\text{sort}} P \log P$, the time is smallest where

$$\frac{dT_A}{dC} = c_{\text{centre}} + c_{\text{score}} \frac{dF_A}{dC} = 0,$$

that is, where opening one more cluster’s worth of routing costs as much as the scoring it saves. The recall curve flattens at high targets (the last few reference documents are spread over many clusters), so F_A stops

Table 6: Clusters P that must be opened, and documents those clusters hold on average, for each recall target and cluster count. These are the F_A values of the time formula.

Clusters C	80%		90%		95%		99%	
	P	docs	P	docs	P	docs	P	docs
64	6	98,610	11	178,239	18	286,892	37	580,487
256	12	50,005	29	118,453	52	211,419	119	480,127
1,024	31	30,626	84	82,157	164	160,584	420	412,040
4,096	72	18,107	211	52,164	467	114,477	1,330	325,353
8,192	127	15,615	353	42,847	778	93,930	2,440	295,858

falling quickly beyond a few thousand clusters, while $c_{\text{centre}}C$ keeps growing. With $c_{\text{score}} \approx 27$ ns per document (Section 4) and a centre comparison of the same order, the formula puts the optimum at $C = 4,096$ or $8,192$ for every target: the difference in F_A between them is worth about 0.7 ms at 99% and only 0.07 ms at 80%, against roughly 0.1 ms of extra routing. Section 4 measures both.

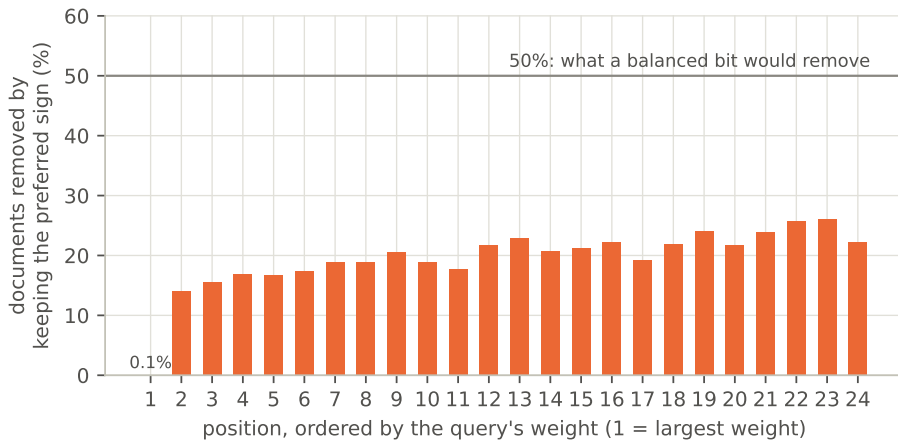
3.2 Bitplanes

Method B starts from the same clusters and adds two settings, the leaf size L and the node budget. At the same C and P the two time formulas differ by

$$T_B - T_A = c_{\text{split}}V_s + c_{\text{leaf}}V_l + c_{\text{group}}G - c_{\text{score}}(F_A - F_B). \quad (2)$$

Method B wins only if the last term, the scoring it avoids, is larger than the three mask terms. Two questions decide the size of $F_A - F_B$: how many documents does one split remove, and may the removed group be skipped at all?

How much a split removes. A split keeps the documents whose bit agrees with the query sign at one position. If document signs were balanced, half of a group would go each way and twelve splits would cut a million documents to about 244. On this dataset the signs are not balanced. Figure 8 shows the share of documents removed by keeping the preferred sign at each position, ordered by the query’s weight. At the position with the largest weight only 99.9% of documents agree with the query and 0.1% are removed; this position carries the same sign in almost every document and query, so it says nothing about which documents are close. At the next eleven positions a split removes between 14% and 22%. After the twelve largest weights, 12.5% of the collection is still in the group, 124,689 documents instead of 244. Inside the clusters opened for 99% recall the effect is weaker still: those documents already resemble the query, and twelve splits keep 19% of them.

**Figure 8:** What one split removes on this dataset, by the rank of the query weight at the split position, averaged over the 200 queries. A balanced bit would remove 50%. Real embeddings agree with the query on its heavy positions far more often than not, so each split removes a small minority.

Whether the removed group may be skipped. Without a budget, method B drops a group only when its penalty exceeds the gap g (Section 2.3). On this dataset the gap averages 15% of the total weight Q (between 9% and 21% across queries), while the largest single weight is only 2.3% of Q and the twelve largest together are 15%. A group that disagrees with the query at one heavy position therefore has a penalty far

below g , and the search must visit it. Adding up the weights in order, a group would need to disagree at about 13 of the heaviest positions before the ceiling rule could drop it, and by then it holds almost nothing. In the exact mode, then, $F_B = F_A$: every document that method A scores, method B scores too, after paying for the masks. Equation 2 is minimised by making no splits at all, $L \geq n_c$, which leaves

$$T_B - T_A = c_{\text{leaf}} \sum_{c \text{ opened}} W_c + c_{\text{group}} P > 0.$$

With a budget. A node budget stops the search early and turns method B into an approximate filter: it scores the groups that agree with the most heavy signs and drops the rest, losing the reference documents in them. Whether that is a good trade depends on how well “agrees with the heavy signs” predicts “is a true neighbour”, compared with the filter method A already has, “is in a nearby cluster”. Figure 9 plots both on the same axes: documents kept against true neighbours kept. Twelve splits keep 12.5% of the documents and 85 of the 100 neighbours; opening the 72 closest of 4,096 clusters keeps 1.8% of the documents and 80 neighbours. Clusters are the sharper filter at every point, so any recall that a budget gives up is recovered more cheaply by opening fewer clusters. Method B with a budget cannot beat method A here either.

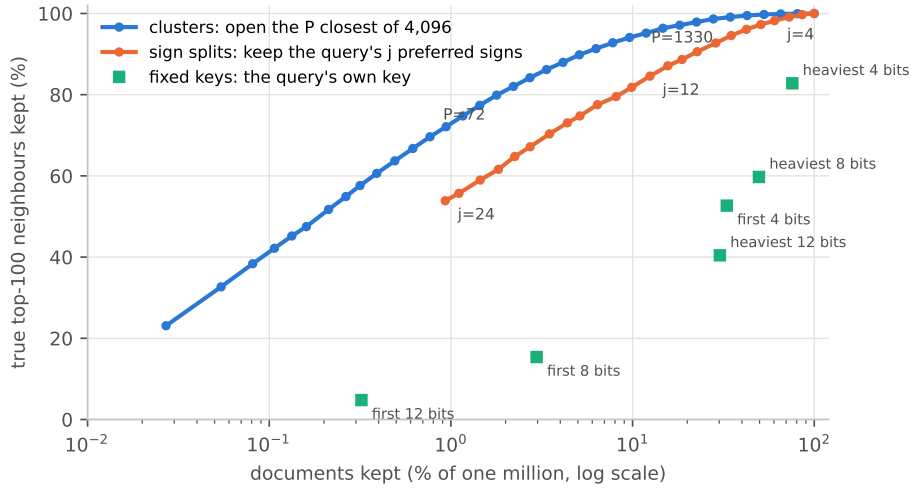


Figure 9: Three ways to keep only part of the collection, on the same dataset. A good filter sits high and to the left: few documents kept, many true neighbours kept. Cluster selection (blue) dominates sign splits (orange) and fixed keys (green) at every level.

When Bitplanes would win. The formula also says what a dataset must look like for the mask work to pay. Two conditions are needed together: a split at a heavy position must remove a large share of the group (balanced signs), and the heaviest few weights must exceed the gap, $\sum_{i \leq j} |q_{(i)}| > g$ for a small j , so that the ceiling drops the other child. When both hold, one path of j splits reads jW words and scores about $n2^{-j}$ documents; with c_{split} near half a nanosecond per word and c_{score} near 27 ns per document, that path is cheaper than scoring the cluster as soon as j is a few units, by a factor that grows like 2^j . Queries whose weight is concentrated in a dozen positions that change from query to query are the case in which method B is designed to shine. This dataset’s queries spread their weight over hundreds of positions.

3.3 Backward Walk

Method C adds the key width h . At the same clusters,

$$T_C - T_A = c_{\text{lookup}} H + c_{\text{key}} E - c_{\text{score}} (F_A - F_C), \quad (3)$$

and the walk skips a key only when its penalty exceeds the gap g . The largest penalty a key can have is the total weight of the h key positions, so the question is whether those fixed positions carry more than 15% of Q . They do not. The first eight positions carry 5.2% of the query weight on average and the first twelve 6.8%; even the twelve positions with the largest average weight across all queries carry only 9.5%. No key is ever skipped, so the walk visits all 2^h keys, $H = 2^h P$, and $F_C = F_A$. The difference is minimised by the narrowest key: $h = 4$ costs $16P$ lookups on top of method A. Any wider key adds lookups without removing scoring.

A limit on scored documents would make method C an approximate filter, like a budget for method B. Figure 9 shows how the query’s own key performs in that role. With the first twelve positions as the key, the query’s

key holds 3,217 documents but only 5 of the 100 true neighbours: a key built from light positions selects documents that agree where it hardly matters. With the twelve heaviest positions, the key holds 302,372 documents and 40 neighbours, because those positions are the ones almost every document shares. Either way the key is a far worse filter than the clusters, so a candidate limit cannot make method C competitive on this dataset.

When Backward Walk would win. Method C needs what method B needs, with one more condition: the heavy positions must be the same for every query, because the key positions are fixed when the index is built. Given that, a directory lookup is cheaper than a mask split, and method C would reach the small group in one lookup where method B needs h splits.

3.4 Comparison

Table 7 collects the conditions and the dataset’s values. Both new methods can only remove scoring work when a small set of query positions is heavy enough to exceed the gap and balanced enough to split the documents. On this dataset neither condition holds, so the formulas predict, for every recall target, the order

$$T_A < T_B \text{ (no splits)} < T_C \text{ (} h = 4 \text{)},$$

with method B and C behind by their bookkeeping only: about $c_{\text{leaf}} \sum W_c + c_{\text{group}} P$ for B and $16 c_{\text{lookup}} P$ for C. At the 99% target with $C = 4,096$ and $P = 1,330$, those are fractions of a millisecond on top of about $c_{\text{score}} \times 325,353 \approx 9$ ms of scoring. Table 8 states where each method is expected to be fastest in general. Section 4 tests the prediction for this dataset.

Table 7: What each new method needs in order to skip scoring work, and what this dataset provides. g is the gap of Table 2, averaged over queries.

Condition	Needed by	This dataset	Met
Heaviest few weights exceed the gap: $\sum_{i \leq j} q_{(i)} > g$ for small j	B and C	12 heaviest weights: 15% of Q ; gap: 15% of Q ; 13 positions needed	no
A split on a heavy position removes about half the group	B and C	removes 0.1% at the heaviest position, 14–22% at the next eleven	no
The heavy positions are the same for every query	C only	first 12 positions: 6.8% of Q ; 12 heaviest on average: 9.5%	no
A cluster boundary is a weak filter, so local pruning has work left to save	B and C	72 of 4,096 clusters hold 80 of 100 neighbours in 1.8% of the documents	no

Table 8: Where each method is expected to be fastest, from the formulas of Section 2.

Fastest	Situation
A: original	Query weight is spread over many positions, or documents mostly share the query’s sign at its heavy positions. No group or key can be dropped, so extra structure only adds work. This dataset.
B: bitplanes	A few heavy positions carry more weight than the gap, their signs split the documents evenly, and which positions are heavy changes from query to query. Splits follow each query’s own order.
C: backward walk	The same as B, but the heavy positions are the same for all queries, so a fixed key captures them and one lookup replaces several splits.

4 Experiments

The experiment has two jobs: check that the code does the work the formulas count, and find which method is fastest at each recall target. Everything is measured on one machine, an Apple M5 Max with 128 GiB of RAM, using one CPU thread and one query at a time.

4.1 Setup

The inputs are fixed and identical for every setting. The documents are one million MS MARCO passages [4], embedded with Nomic Embed v1.5 [3], cut to the first 256 values and stored as 256 sign bits. The 200 queries come from the same collection’s development set. The reference answer for each query is its exact top 100

when all one million documents are scored. The arrays are read in place; their hashes are recorded before and after the run and did not change.

What varies is only the index. We use the cluster counts $C \in \{64, 256, 1024, 4096, 8192\}$, with cluster centres learned once by k -means [5], plus the whole collection as a single cluster. For each C , P takes the four values from Table 6, one per recall target. Every method is measured on all 21 resulting (C, P) pairs, so no method gets a routing advantage. Within each pair, method A has no further settings; method B is run without splits, with leaf size 128 and no budget, and with leaf size 128 and node budgets of 64, 512 and 4,096; method C is run with key widths 4, 8 and 12. That gives 189 settings. Each runs in its own process, builds its index from the same codes, answers the 200 queries twice (once for the slowest setting), and records for every query the time, the returned IDs and the operation counts of Section 2. We report the median time over the 400 timings and the mean recall over the 200 queries. The largest process peak was 1.5 GB, far below the 32 GB limit, so every setting is eligible.

4.2 Checking the calculations

Table 9 compares the counts that Section 2 predicts from the data and the settings with the counts the code recorded. Every one matches: the documents scored equal the documents in the opened clusters, the recall of an exact setting equals the routing recall r_{route} , method B without splits reads exactly $\sum_c W_c$ leaf words, method C visits all 2^h keys and makes $2^h P$ lookups, and the index sizes are the byte counts of Section 2.2. The work formulas are therefore a correct description of the code.

Table 9: Predicted against recorded counts over the 189 settings. “None” means every setting agreed exactly.

Quantity (formula)	Settings checked	Largest difference
Documents scored, F (exact settings)	126	0.00%
Recall = r_{route} (exact settings)	126	none
Leaf words V_l (B without splits)	21	none
Split words V_s (B without splits)	21	none
Keys visited $E = 2^h$	63	none
Lookups $H = 2^h P$	63	none
Code bytes = $32N$	189	none
ID bytes = $8N$	189	none
Bitplane bytes = $8d \sum_c W_c$	105	none
Directory bytes = $20U$	63	none

Turning counts into time needs the constants $c_{\cdot}$. We fit them by least squares on the measured search times of each method’s settings, weighting each setting by its own time so that fast and slow settings count equally (Table 10). Scoring one document costs about 0.026 μs in all three methods, a directory lookup about 13 ns, a split word well under a nanosecond, and taking a group from the queue about a quarter of a microsecond. Figure 10 plots the time the formula gives against the time measured. For method A the median error is 1.7%, for method C 8.0%, and for method B without splits 11%.

Table 10: Unit costs from the fit, in microseconds, and the median absolute error of the fitted formula over each method’s settings.

Method	Cost	Microseconds	Settings	Median error
A: original	per call, c_{call}	45.8228	21	1.7%
	per scored document, c_{score}	0.0260		
B: bitplanes	per call, c_{call}	9.2597	105	13.7%
	per scored document, c_{score}	0.0295		
	per split word, c_{split}	0.0003		
	per leaf word, c_{leaf}	0.0032		
	per group taken from the queue, c_{group}	0.2394		
C: backward walk	per call, c_{call}	11.0438	63	8.0%
	per scored document, c_{score}	0.0308		
	per directory lookup, c_{lookup}	0.0132		
	per key generated, c_{key}	0.0000		

The formula is least accurate for method B when it splits wide clusters. At $C = 64$ with 37 opened clusters and leaf size 128 the measured time is 72 ms against 35 ms from the formula, and the single-cluster search with leaf size 128 takes 745 ms against 291 million split words and 36,983 groups. The counts are right; the

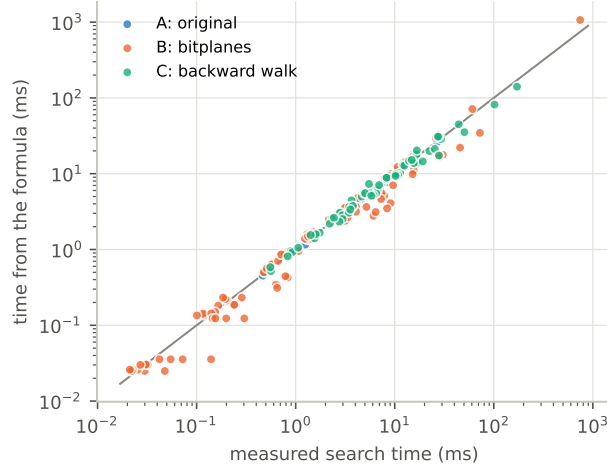


Figure 10: Search time computed from the recorded counts and the unit costs of Table 10, against the measured median. Points on the grey line are exact.

constants are not constant. When many groups wait in the queue, their masks and the bitplanes no longer fit in the processor caches, and each word costs several times more than in the small-cluster settings that dominate the fit. The error runs in one direction only: splitting is more expensive than the formula says, so it does not change the comparison below.

4.3 Results

Table 11 and Figure 11 give, for each recall target, the fastest setting of each method that reached the target. Method A is fastest at all four targets. Method B is behind by 1% to 4%, and its fastest setting at every target is the one without splits, as Section 3.2 derived. Method C is behind by 9% to 28% with its narrowest key, $h = 4$, as Section 3.3 derived. All three methods score the same documents at the same (C, P) ; the differences are the bookkeeping that Section 3.4 predicted, about $0.4 \mu\text{s}$ per opened cluster for B and 16 lookups per opened cluster for C.

Table 11: Fastest setting of each method at each recall target, over all measured settings that reached the target. Recall is the mean over 200 queries; time is the median per query including cluster selection.

Target	Method	Setting	Recall	Median ms	Docs scored
80%	A: original	$C = 4,096, P = 72$	80.01%	0.569	18,107
	B: bitplanes	$C = 4,096, P = 72$; no splits	80.01%	0.576	18,107
	C: backward walk	$C = 4,096, P = 72$; $h = 4$	80.01%	0.621	18,107
90%	A: original	$C = 8,192, P = 353$	90.02%	1.442	42,847
	B: bitplanes	$C = 8,192, P = 353$; no splits	90.02%	1.496	42,847
	C: backward walk	$C = 8,192, P = 353$; $h = 4$	90.02%	1.741	42,847
95%	A: original	$C = 8,192, P = 778$	95.00%	2.872	93,930
	B: bitplanes	$C = 8,192, P = 778$; no splits	95.00%	3.022	93,930
	C: backward walk	$C = 8,192, P = 778$; $h = 4$	95.00%	3.693	93,930
99%	A: original	$C = 4,096, P = 1,330$	99.00%	8.702	325,353
	B: bitplanes	$C = 8,192, P = 2,440$; no splits	99.00%	9.023	295,858
	C: backward walk	$C = 8,192, P = 2,440$; $h = 4$	99.00%	11.130	295,858

The other settings show the rest of the prediction. For method A, $C = 4,096$ and $8,192$ are within 1% of each other at 99% (8.70 and 8.78 ms), while 64 clusters take 14.9 ms: fewer clusters open more documents, as Table 6 says. Cluster selection itself costs 0.07 ms for 4,096 centres and 0.14 ms for 8,192, growing to 0.37 ms when 2,440 of them must be sorted out, which is why 4,096 wins at 80% and 99% and 8,192 at 90% and 95%. For method B, splitting always costs more than it saves: with leaf size 128 and no budget the 99% setting takes 11.1 ms instead of 9.0, and the 80% setting 0.79 ms instead of 0.58, because $F_B = F_A$ and the masks are pure overhead. With a budget the search is faster but loses most of the answer: at $C = 8,192, P = 2,440$ a budget of 4,096 groups reaches 44% recall in 5.5 ms and a budget of 512 reaches 37% in 1.4 ms, while method A reaches 80% in 0.57 ms by opening fewer clusters. The best budgeted setting that reaches 80% takes 0.72 ms, still behind A. For method C, every extra key bit doubles the lookups: at the 99% clusters

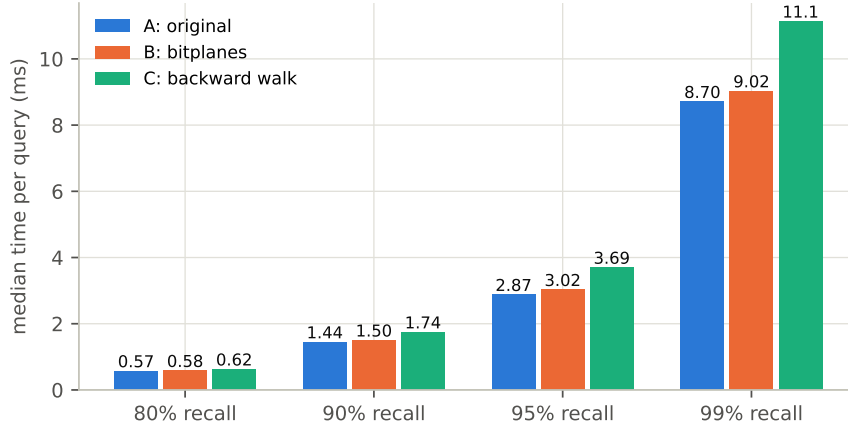


Figure 11: The same fastest settings as Table 11. Lower is better.

$h = 4, 8$ and 12 take 11.1, 28.3 and 172 ms. Figure 12 shows all 189 settings together.

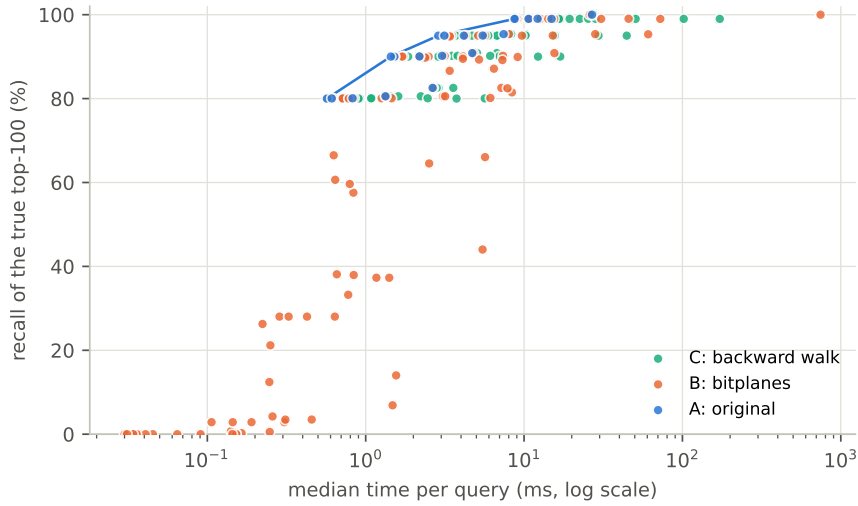


Figure 12: Every measured setting. The blue line joins method A’s fastest settings at the four targets. Orange points below 80% are budgeted bitplane searches; green points beyond 20 ms are backward walks with wide keys.

4.4 Code references

The search code is `cpp/index.cpp` (method A in `Index::scan`, method B in `Index::search`), `cpp/key_index.cpp` (method C) and `cpp/score.hpp` (the shared lookup-table scorer, top- K heap and ceiling rule). The experiment is `experiments/fixed_data_study.py`; each setting is measured by `experiments/worker.py`. The run, with every per-query record, is under `results/fixed-data-2026-09-12/`: `settings.csv` has one row per setting, `selected.csv` the fastest per method and target, `checks.json` the count comparisons and `unit-costs.json` the fit. To repeat the run on this machine:

```
.venv/bin/python -m experiments.fixed_data_study run --output results/my-run
.venv/bin/python reports/search-fixed-data/make_results.py facts
.venv/bin/python reports/search-fixed-data/make_results.py results --run results/my-run
bash reports/search-fixed-data/build.sh
```

5 Limitations and next steps

The result holds for one dataset, one embedding model and one code width. The two properties that decided it, how much weight a query puts on its heaviest positions and how often documents share the query’s sign there, are properties of the embedding model. A model whose queries concentrate their weight in a few

positions, or a code whose bits are closer to balanced, would move the dataset in Table 7 and could change the winner. The natural next step is to measure those two properties for other embedding models before building any index; both take minutes on a sample and Section 3 shows how to read them.

The time formula for method B is exact in its counts but optimistic in its constants when clusters are wide, because the per-word cost depends on cache behaviour. A model that prices cold masks separately would fit better; it would make splitting look worse, not better. The single-thread, single-query measurement also leaves out batching and multi-core effects that a production system would have.

Some design choices were not explored. Method C used contiguous key positions; choosing the positions with the largest average weight is possible but, on this dataset, still leaves the key well below the gap. Method B could store sparse masks instead of full-width ones, which would lower its constant but not create documents to skip. Finally, all results are for $K = 100$; a smaller K makes the gap smaller and pruning slightly easier, but not by enough to cross the threshold here.

6 Conclusion

We compared Exa’s cluster-then-scan search with two additions that try to skip scoring work, on one fixed dataset with only the index parameters varied. The calculations of Section 2 describe the code exactly: every predicted operation count matched the recorded one. The derivation of Section 3 predicted that neither addition can skip anything on this dataset, because a single sign bit removes only 14% to 22% of the documents and no group or key ever exceeds the gap that would let the ceiling rule drop it. The measurements of Section 4 confirmed the prediction: the original method was fastest at every recall target, Bitplanes was 1% to 4% slower with its best setting being “no splits”, and Backward Walk was 9% to 28% slower with its narrowest key.

The methods are not useless; they are aimed at a different kind of data. Bitplanes win when a few heavy query positions exceed the gap and split the documents evenly, and Backward Walk wins when, in addition, those positions are the same for every query. The formulas give those conditions in a form that can be checked on a sample before an index is built, which is the practical outcome of this study.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024. <https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. NeurIPS 2022. <https://arxiv.org/abs/2205.13147>.
- [3] Z. Nussbaum, J. X. Morris, B. Duderstadt and A. Mulyar. *Nomic Embed: Training a Reproducible Long Context Text Embedder*. 2024. <https://arxiv.org/abs/2402.01613>.
- [4] P. Bajaj et al. *MS MARCO: A Human Generated MACHine Reading COMprehension Dataset*. 2016. <https://arxiv.org/abs/1611.09268>.
- [5] M. Douze et al. *The Faiss library*. 2024. <https://arxiv.org/abs/2401.08281>.